What is a Binary Search Tree (BST)?

A Binary Search Tree is a node-based binary tree data structure that follows these strict rules:

1. **Left Subtree:** Contains only nodes with keys **less than** the node's key.
2. **Right Subtree:** Contains only nodes with keys **greater than** the node's key.
3. **Recursive Nature:** The left and right subtrees must also be binary search trees.

Advantages of BST

- **Easy Searching:** You always have a "hint" of where to go. If the value you want is smaller than the current node, go left; if larger, go right.
- **Efficiency:** Compared to arrays and linked lists, insertion and deletion operations are generally much faster in a BST.

BST Operations: Insertion

To insert a node, you start at the root and compare values.

- If the new value is smaller than the current node, move to the **left** child.
- If the new value is larger, move to the **right** child.
- Repeat until you find an empty (**NULL**) spot to place the node.

C Implementation of BST

Your slides provide a complete program to build and display a BST. Note that I've fixed a few typos found in the slide images (like `inorederTraversal` and a pointer assignment error in `createNode`) to ensure the code is functional.

1. Node Structure and Creation

C

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};
```

```c
// Function to create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
```

2. Recursive Insertion Function

C

```c
// Function to insert an element into BST
struct Node* insert(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }

    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    }

    return root;
}
```

3. Displaying the Tree (Inorder Traversal)

As we learned earlier, an **Inorder Traversal** of a BST will always print the values in sorted (ascending) order.

C

```c
void inorderTraversal(struct Node* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}
```

4. Main Function

```
C
```

```c
int main() {
    struct Node* root = NULL;

    // Building the tree from your example
    root = insert(root, 50);
    insert(root, 20);
    insert(root, 30);
    insert(root, 70);
    insert(root, 90);

    printf("Values in sorted order (Inorder): \n");
    inorderTraversal(root);

    return 0;
}
```

Deleting a node from a Binary Search Tree (BST) is slightly more complex than insertion because you must maintain the BST properties after the node is gone. The approach depends on how many children the node has.

The Three Deletion Scenarios

1. Case 1: Node to be deleted is a Leaf (No Children)

This is the simplest case. You just remove the node and set the parent's pointer to `NULL`.

2. Case 2: Node has One Child

You "bypass" the node. The child of the deleted node is linked directly to the deleted node's parent.

3. Case 3: Node has Two Children

This is the trickiest part. You cannot simply delete the node because it would leave two "orphaned" subtrees. Instead:

1. Find the **Inorder Successor** (the smallest node in the right subtree).
2. Copy the Inorder Successor's value to the current node.
3. Delete the Inorder Successor (which is guaranteed to have at most one child).

C Implementation: BST Deletion

To handle Case 3, we first need a small helper function to find the minimum value in a subtree.

Helper: Find Minimum Value Node

C

```
struct Node* minValueNode(struct Node* node) {
    struct Node* current = node;
    // Loop down to find the leftmost leaf
    while (current && current->left != NULL)
        current = current->left;
    return current;
}
```

The Deletion Function

C

```
struct Node* deleteNode(struct Node* root, int key) {
    // Base Case: Tree is empty
    if (root == NULL) return root;

    // 1. Navigate to the node
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);

    // 2. Node found! Prepare for deletion
    else {
        // Cases 1 & 2: One child or no child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
```

```
        // Case 3: Two children
        // Get the inorder successor (smallest in the right subtree)
        struct Node* temp = minValueNode(root->right);

        // Copy the successor's data to this node
        root->data = temp->data;

        // Delete the inorder successor
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}
```

How it works in main()

You can test the deletion by modifying your previous code:

C

```
int main() {
    struct Node* root = NULL;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 20);
    insert(root, 40);
    insert(root, 70);
    insert(root, 60);
    insert(root, 80);

    printf("Original tree (Inorder): ");
    inorderTraversal(root); // Output: 20 30 40 50 60 70 80

    printf("\nDeleting 20 (Leaf)...\n");
    root = deleteNode(root, 20);
    inorderTraversal(root);

    printf("\nDeleting 30 (Node with one child)...\n");
    root = deleteNode(root, 30);
    inorderTraversal(root);

    printf("\nDeleting 50 (Root with two children)...\n");
    root = deleteNode(root, 50);
    inorderTraversal(root);

    return 0;
```

```
}
```

Peers's Tip: When deleting a node with two children, you can also use the **Inorder Predecessor** (the largest node in the left subtree) instead of the successor. Both methods are valid and maintain the BST structure perfectly!

Searching in a Binary Search Tree (BST) is highly efficient because the tree's structure allows you to eliminate half of the remaining nodes at every step. This results in an average time complexity of $O(\log n)$.

The Search Logic

1. **Base Case:** If the current node is `NULL`, the key does not exist in the tree. If the current node's data matches the key, you've found it!
2. **Recursive Step (Left):** If the key is **smaller** than the current node's data, the search continues in the left subtree.
3. **Recursive Step (Right):** If the key is **larger** than the current node's data, the search continues in the right subtree.

C Implementation

You can implement this search using either **recursion** (more intuitive) or an **iterative** approach (more memory-efficient as it avoids stack overhead).

1. Recursive Approach

C

```
struct Node* search(struct Node* root, int key) {
    // Base Cases: root is null (not found) or key is present at root
    if (root == NULL || root->data == key)
        return root;

    // Key is greater than root's data
    if (key > root->data)
        return search(root->right, key);

    // Key is smaller than root's data
    return search(root->left, key);
}
```

2. Iterative Approach

C

```
struct Node* searchIterative(struct Node* root, int key) {
```

```
    while (root != NULL && root->data != key) {
        if (key < root->data)
            root = root->left;
        else
            root = root->right;
    }
    return root;
}
```

How to Use it in `main()`

```
C

int main() {
    // Assuming 'root' is already populated with 50, 30, 70, 20, 40, 60, 80
    int target = 40;
    struct Node* result = search(root, target);

    if (result != NULL)
        printf("Node %d found in the BST.\n", result->data);
    else
        printf("Node %d not found.\n", target);

    return 0;
}
```

Performance Note

In a balanced BST, the number of comparisons is equal to the **height** of the tree. This is why keeping a tree balanced is so important—if the tree becomes a "skewed" line, your search speed drops from the lightning-fast $O(\log n)$ to a sluggish $O(n)$, which is no better than a standard linked list!

To find the **height** and **depth** of a Binary Search Tree (BST), we look at how far nodes are from the root or the leaves. While the terms are related, they represent different perspectives of the tree's "size."

---

1. Depth of a Node

The **depth** of a node is the number of edges from the **root** to that specific node.

- The **Root** always has a depth of **0**.
- Each step down to a child increases the depth by 1.

C Implementation: Depth of a Key

To find the depth of a specific value in a BST, we search for the node and count the steps.

```C
int findDepth(struct Node* root, int x) {
    if (root == NULL) return -1; // Node not found

    int dist = 0;
    struct Node* current = root;

    while (current != NULL) {
        if (x == current->data) return dist;

        dist++;
        if (x < current->data)
            current = current->left;
        else
            current = current->right;
    }
    return -1; // Not found
}
```

2. Height of a Node

The **height** of a node is the number of edges on the **longest path** from that node to a **leaf.**

- A **Leaf** node always has a height of **0.**
- The height of the **Root** is the height of the entire tree.

C Implementation: Height of a Tree

To find the height of the entire tree, we recursively calculate the height of the left and right subtrees and take the maximum.

```C
int findHeight(struct Node* node) {
    if (node == NULL) {
        // Technically, a NULL node has a height of -1
        // so that a leaf node (NULL children) ends up with 0.
        return -1;
    }

    int leftHeight = findHeight(node->left);
```

```
    int rightHeight = findHeight(node->right);

    // Height = 1 + max(left_subtree, right_subtree)
    if (leftHeight > rightHeight)
        return leftHeight + 1;
    else
        return rightHeight + 1;
}
```

Key Differences at a Glance

Feature	Depth	Height
Starting Point	The Root	The Leaves
Direction	Downward (Root $\rightarrow$ Node)	Upward (Leaf $\rightarrow$ Node)
Root Value	0	Maximum (Max levels - 1)
Leaf Value	Maximum	0

Finding the minimum and maximum elements in a Binary Search Tree (BST) is very efficient because of the tree's sorted property.

The Logic

- **Minimum Value:** Since all smaller values are in the left subtree, the smallest element is the **leftmost node.** You simply start at the root and keep moving to the left child until you can't go any further.

- **Maximum Value:** Since all larger values are in the right subtree, the largest element is the **rightmost node.** Start at the root and keep moving to the right child until you can't go any further.

C Implementation

These functions take the root of the tree as an argument and return the node containing the minimum or maximum value.

1. Finding the Minimum Element

C

```
struct Node* findMin(struct Node* root) {
    if (root == NULL) return NULL; // Empty tree

    struct Node* current = root;
    // Go to the leftmost node
    while (current->left != NULL) {
        current = current->left;
    }
    return current;
}
```

2. Finding the Maximum Element

C

```
struct Node* findMax(struct Node* root) {
    if (root == NULL) return NULL; // Empty tree

    struct Node* current = root;
    // Go to the rightmost node
    while (current->right != NULL) {
        current = current->right;
    }
```

```
    return current;
}
```

Integration in `main()`

You can call these functions to find the range of your data:

```
C

int main() {
    // Assuming root is already populated
    struct Node* minNode = findMin(root);
    struct Node* maxNode = findMax(root);

    if (minNode != NULL)
        printf("The minimum key is: %d\n", minNode->data);

    if (maxNode != NULL)
        printf("The maximum key is: %d\n", maxNode->data);

    return 0;
}
```

Efficiency Note

In a balanced tree, both operations take **$O(\log n)$** time. In the worst-case scenario (a skewed tree that is just a straight line to the left or right), it takes **$O(n)$** time. Unlike searching for a random key, you never have to "backtrack" or check the other side of the tree!

Finding the **Inorder Predecessor** and **Inorder Successor** is essential for operations like node deletion and navigating through sorted data in a BST.

The Definitions

- **Inorder Predecessor:** The node with the largest value that is smaller than the given node (the node that comes immediately before in an inorder traversal).
- **Inorder Successor:** The node with the smallest value that is larger than the given node (the node that comes immediately after in an inorder traversal).

The Logic

Case 1: The node has a left/right subtree

- **Predecessor:** If the node has a left subtree, the predecessor is the **maximum** value in that left subtree (leftmost of the right).
- **Successor:** If the node has a right subtree, the successor is the **minimum** value in that right subtree.

Case 2: The node has NO left/right subtree

- We must search from the root. For the successor, the last node where we took a **left turn** to find our target is the successor. For the predecessor, it is the last node where we took a **right turn.**

C Implementation

These functions use a search-based approach to find the predecessor and successor of a specific key.

C

```c
#include <stdio.h>
#include <stdlib.h>

// Helper to find the maximum value in a subtree
struct Node* findMax(struct Node* root) {
    while (root->right != NULL) root = root->right;
    return root;
}

// Helper to find the minimum value in a subtree
struct Node* findMin(struct Node* root) {
    while (root->left != NULL) root = root->left;
    return root;
}

// Function to find Predecessor and Successor
```

```
void findPreSuc(struct Node* root, struct Node** pre, struct Node** suc, int key) {
    if (root == NULL) return;

    // If key is found at root
    if (root->data == key) {
        // Predecessor is the maximum value in left subtree
        if (root->left != NULL)
            *pre = findMax(root->left);

        // Successor is the minimum value in right subtree
        if (root->right != NULL)
            *suc = findMin(root->right);
        return;
    }

    // If key is smaller than root, go left
    if (key < root->data) {
        *suc = root; // Root could be the successor
        findPreSuc(root->left, pre, suc, key);
    }
    // If key is larger than root, go right
    else {
        *pre = root; // Root could be the predecessor
        findPreSuc(root->right, pre, suc, key);
    }
}
```

How to Use it in `main()`

```
C
```

```
int main() {
    struct Node* pre = NULL;
    struct Node* suc = NULL;
    int target = 6;

    // Build tree as per your slide: 6, 2, 9, 1, 5, 8, 4, 7
    findPreSuc(root, &pre, &suc, target);

    if (pre != NULL)
        printf("Predecessor of %d is %d\n", target, pre->data);
    else
        printf("No Predecessor\n");

    if (suc != NULL)
        printf("Successor of %d is %d\n", target, suc->data);
    else
        printf("No Successor\n");

    return 0;
}
```

Efficiency Note

Like search, this operation runs in **$O(h)$** time, where h is the height of the tree. In a balanced tree, this is **$O(\log n)$.**

Verifying if a binary tree is a **Binary Search Tree (BST)** is a classic problem. A common mistake is just checking if a node is greater than its left child and smaller than its right child. However, that isn't enough—**every** node in the left subtree must be smaller than the root, and **every** node in the right subtree must be larger.

As seen in your slide's example, node **11** is larger than its parent **5**, but it violates the BST property because it's in the left subtree of **6**, meaning it must be smaller than **6.**

The Logic: Range Constraints

To correctly verify a BST, we pass down a **range** (minimum and maximum allowed values) for each node:

- When moving **left**, we update the **maximum** allowed value to `node->data - 1`.
- When moving **right**, we update the **minimum** allowed value to `node->data + 1`.

C Implementation

This function uses recursion and handles the constraints by using `INT_MIN` and `INT_MAX` from `<limits.h>` for the initial root call.

C

```c
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

// Function to check if the tree is a BST within a given range
bool isBSTUtil(struct Node* node, int min, int max) {
    // 1. An empty tree is a BST
    if (node == NULL) {
        return true;
    }

    // 2. Check if the current node violates the range constraints
    if (node->data < min || node->data > max) {
        return false;
    }

    // 3. Check subtrees recursively with updated ranges
    // Left subtree: max becomes node->data - 1
    // Right subtree: min becomes node->data + 1
    return isBSTUtil(node->left, min, node->data - 1) &&
           isBSTUtil(node->right, node->data + 1, max);
```

```
}

// Wrapper function
bool isBST(struct Node* root) {
    return isBSTUtil(root, INT_MIN, INT_MAX);
}
```

Alternative Method: Inorder Traversal

Another clever way to verify a BST is to perform an **Inorder Traversal**.

- If the tree is a BST, the values must appear in **strictly increasing order.**
- You can store the value of the previously visited node and compare it with the current one.

```
C

int prevVal = INT_MIN;

bool isBSTInorder(struct Node* root) {
    if (root == NULL) return true;

    // Check left subtree
    if (!isBSTInorder(root->left)) return false;

    // Check current node against previous value
    if (root->data <= prevVal) return false;
    prevVal = root->data;

    // Check right subtree
    return isBSTInorder(root->right);
}
```

Why this matters

Verifying the BST property ensures that your search, insert, and delete operations—which rely on the tree being sorted—actually work correctly. If a tree "breaks" its BST property (like the node **11** in your slide), a search for **11** starting from the root **6** would fail because the algorithm would only look in the right subtree!

Finding the **Lowest Common Ancestor (LCA)** of two nodes n_1 and n_2 is much easier in a Binary Search Tree (BST) than in a regular binary tree because we can use the node values to decide which direction to move.

The LCA is the lowest node in the tree that has both n_1 and n_2 as descendants.

The Logic

When searching for the LCA of two values n_1 and n_2, start at the root and compare their values:

1. **Both values are smaller than the root:** The LCA must be in the **left subtree.** Move to the left child and repeat.
2. **Both values are larger than the root:** The LCA must be in the **right subtree.** Move to the right child and repeat.
3. **Values "split" or one is the root:** If one value is smaller than the root and the other is larger (or if the root matches one of the values), then the **current root is the LCA.**

C Implementation

This recursive function is highly efficient as it only visits nodes along a single path from the root to the LCA.

C

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};
```

```
// Function to find the LCA of two nodes n1 and n2
struct Node* findLCA(struct Node* root, int n1, int n2) {
    if (root == NULL) return NULL;

    // 1. If both n1 and n2 are smaller than root, LCA is in left
    if (root->data > n1 && root->data > n2) {
        return findLCA(root->left, n1, n2);
    }

    // 2. If both n1 and n2 are larger than root, LCA is in right
    if (root->data < n1 && root->data < n2) {
        return findLCA(root->right, n1, n2);
    }

    // 3. We have found the split point. This is the LCA.
    return root;
}
```

Explanation of Slide Examples

Using the logic above on the tree from your slide:

- **LCA(4, 5):** Starting at **1**, both are in the left subtree**.** Move to **2.** At **2**, 4 is smaller and 5 is larger**. Split found! LCA = 2.**
- **LCA(4, 6):** Starting at **1**, 4 is smaller (left) and 6 is larger (right)**. Split found immediately! LCA = 1.**
- **LCA(2, 4):** Starting at **1**, both are smaller**.** Move to **2. 2** matches one of the nodes**. LCA = 2.**

Complexity Analysis

- **Time Complexity: $O(h)$**, where h is the height of the tree. In a balanced BST, this is **$O(\log n)$.**
- **Space Complexity: $O(h)$** for the recursion stack (or $O(1)$ if implemented iteratively)**.**

Converting a Binary Search Tree (BST) into a **sorted Doubly Linked List (DLL)** in-place is a classic challenge. The key is to leverage an **Inorder Traversal** (or a reverse Inorder traversal, as shown in your "Steps" slide) to ensure the nodes are linked in their sorted order.

By doing this "in-place," we reuse the `left` pointer as the `prev` pointer and the `right` pointer as the `next` pointer of the Doubly Linked List.

The Logic (Reverse Inorder Approach)

As outlined in your second slide, the goal is to build the list from the "back" to the "front" (largest to smallest):

1. **Recur for the right subtree:** Process the largest elements first.
2. **Modify pointers:**
    - Set the current node's `right` to point to the current `head` of the DLL.
    - If the `head` is not NULL, set the `head`'s `left` to point back to the current node.
    - Move the `head` to the current node.
3. **Recur for the left subtree:** Process the smaller elements.

C Implementation

Here is the code based on the logic described in your slides:

C

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

// This pointer will keep track of the head of the DLL
struct Node* head = NULL;

void BSTToDLL(struct Node* root) {
    // Base Case
    if (root == NULL) return;
```

```
    // 1. Recur for the right subtree (Reverse Inorder)
    BSTToDLL(root->right);

    // 2. Make current node point to the current head
    root->right = head;

    // 3. Change left pointer of the current head
    if (head != NULL) {
        head->left = root;
    }

    // 4. Move head to the current node (building from back to front)
    head = root;

    // 5. Recur for the left subtree
    BSTToDLL(root->left);
}

// Function to print the Doubly Linked List
void printList(struct Node* node) {
    printf("Sorted Doubly Linked List: \n");
    while (node != NULL) {
        printf("%d ", node->data);
        node = node->right;
    }
}
```

Alternative: Standard Inorder Approach

You can also build the list from "front" to "back" (smallest to largest) using a prev pointer:

```C
struct Node* prev = NULL;
struct Node* dllHead = NULL;


void convertToDLL(struct Node* root) {
    if (root == NULL) return;

    convertToDLL(root->left);

    // If this is the first node (smallest), mark it as head
    if (prev == NULL) {
        dllHead = root;
    } else {
        root->left = prev;
        prev->right = root;
    }
    prev = root;

    convertToDLL(root->right);
}
```

Complexity Analysis

- **Time Complexity: $O(n)$**, as every node is visited exactly once.
- **Space Complexity: $O(h)$**, where h is the height of the tree, due to the recursion stack.

Finding all nodes within a specific range $[L, R]$ in a Binary Search Tree is a very efficient operation. Instead of visiting every single node, we use the BST properties to "prune" subtrees that we know cannot possibly contain values within that range.

The Logic

We perform a modified Inorder Traversal:

1. **Check Left Subtree:** If the current node's value is **greater than** the low end (L), there might be relevant nodes in the left subtree.
2. **Check Current Node:** If the current node's value is **within** the range $[L, R]$, we process it (print it or add it to a list).
3. **Check Right Subtree:** If the current node's value is **smaller than** the high end (R), there might be relevant nodes in the right subtree.

C Implementation

This function prints all nodes that fall within the range `low` and `high`.

C

```c
#include <stdio.h>



struct Node {

    int data;

    struct Node *left, *right;

};



// Function to print all nodes in range [low, high]

void printRange(struct Node* root, int low, int high) {

    if (root == NULL) return;
```

```
// 1. If root->data is greater than low,
// then there might be nodes in the left subtree
if (low < root->data) {
    printRange(root->left, low, high);
}

// 2. If root->data is in range, print it
if (low <= root->data && high >= root->data) {
    printf("%d ", root->data);
}

// 3. If root->data is smaller than high,
// then there might be nodes in the right subtree
if (high > root->data) {
    printRange(root->right, low, high);
```

```
    }

}
```

Complexity Analysis

- **Time Complexity:** $O(h + k)$, where h is the height of the tree and k is the number of nodes actually in the range. This is much faster than $O(n)$ because we skip irrelevant subtrees.
- **Space Complexity:** $O(h)$ for the recursion stack.

Based on your uploaded image, here is the transcription of the **Applications of Binary Search Trees and General Trees**:

Real-World Applications

1. Binary Search Trees (BST) for Searching

BSTs provide highly efficient searching operations. In a balanced tree, the time complexity is **$O(\log n)$** on average, making them much faster than linear data structures like arrays for large datasets.

2. Expression Trees for Evaluation

Trees can represent mathematical expressions (where internal nodes are operators and leaves are operands). This structure enables compilers and calculators to perform efficient evaluation, simplification, and conversion between infix, prefix, and postfix notations.

3. File Systems and Directories

Operating systems use tree structures to represent hierarchical file systems. In this model:

- **Directories** are internal nodes.
- **Files** are leaf nodes.
- This allows for intuitive navigation and efficient path searching.

4. Network Routing Tables

Trees (specifically "Tries" or prefix trees) help organize routing information in computer networks. This organization allows for fast IP address lookups and efficient data transmission across complex network topologies.

5. Hierarchical Data Representation

Trees are the natural choice for representing any data with a "parent-child" relationship, such as:

- **Organizational Charts:** Visualizing corporate hierarchies.
- **Family Trees:** Mapping genealogical relationships.
- **HTML/XML DOM:** Representing the structure of web pages or data files.